

Student Mental Health Monitoring System

SE LAB Experiment-08 – Part B

Object Design Document (ODD)

Design Patterns, Attributes, Contracts & Invariants

Submitted By:

Mokshi Shah — 241071068

Vanshika Shah — 241071069

April 21, 2026

BATCH-D

Contents

1 Design Patterns Applied to SMHMS	3
1.1 Creational Patterns	3
1.1.1 Pattern 1: Factory Method – Notification Creation ...	3
1.1.2 Pattern 2: Singleton – System Configuration	4
1.2 Structural Patterns	4
1.2.1 Pattern 3: Facade – Analytics Subsystem	4
1.2.2 Pattern 4: Decorator – Role-Based Dashboard Panels ..	5
1.3 Behavioural Patterns	6
1.3.1 Pattern 5: Observer – Alert Notification	6
1.3.2 Pattern 6: Strategy – Risk Scoring Algorithm	7
1.3.3 Pattern 7: State – Counselling Session Lifecycle	7
1.3.4 Pattern 8: Command – Admin Configuration Changes ...	8
2 Missing Attributes and Operations Identified	10
3 Type, Signatures, and Visibility Specification	13
4 Preconditions and Postconditions	19
5 Invariants	21
5.1 Class Invariants	22
5.2 Method-Level Invariants	23
5.3 System-Wide Invariants	24
6 Conclusion	25

1 DESIGN PATTERNS APPLIED TO SMHMS

Design patterns are reusable solutions to recurring design problems. Following the ARENA case study (Bruegge & Dutoit, Ch.9), patterns are organised into three classical GoF categories — Creational, Structural, and Behavioural — and applied to SMHMS to address extensibility, decoupling, state management, and concurrency challenges.

1.1 Creational Patterns

1.1.1 Pattern 1: Factory Method – Notification Creation

Problem: The NotifController must create different notification objects (RiskAlertNotification, SessionReminder, ScreeningReminder, WellnessCampaign) depending on the triggering event, but should not be tightly coupled to concrete notification classes. Adding a new notification type (e.g., crisis SMS) should not require modifying NotifController.

Solution: An abstract NotificationFactory declares createNotification(type, context): Notification. Concrete factories (AlertNotificationFactory, ReminderNotificationFactory) override this method to instantiate the correct subtype. NotifController calls the factory method without knowing the concrete class.

Component	Role	Key Method
NotificationFactory (abstract)	Abstract Creator	createNotification(type, ctx): Notification
AlertNotificationFactory	Concrete Creator	createNotification(...): RiskAlertNotif
ReminderNotificationFactory	Concrete Creator	createNotification(...): Reminder
Notification (abstract)	Abstract Product	deliver(), getRecipient()
NotifController	Client	calls factory; unaware of concrete class

Benefit: Adding a new notification type (e.g., WhatsApp alert) requires only a new factory subclass; NotifController and all existing factories are unmodified. Satisfies the Open/Closed Principle.

1.1.2 Pattern 2: Singleton – System Configuration

Problem: SystemConfig stores risk-scoring threshold weights, RBAC policies, data-retention periods, and notification frequency caps. Multiple controllers (RiskController, NotifController, AuthController) read these at runtime. If multiple instances existed, configuration inconsistencies would arise — e.g., one controller scoring with High ≥ 70 while another uses High ≥ 80 .

Solution: SystemConfig implements the Singleton pattern. A private class-level _instance variable holds the sole object. A thread-safe getInstance() class method (double-checked locking) returns the existing instance or creates it on first call. All mutable state is protected by a reentrant lock to prevent race conditions during concurrent configuration reads/writes.

Attribute / Method	Notes
– _instance: SystemConfig = None [class]	Singleton holder
– riskThresholds: dict	{Low:40, Medium:70, High:100}
– retentionDays: int	Default 730 (2 years)
– rbacPolicies: dict	Role → permission mapping
– _lock: RLock	Thread-safe reentrant lock

Attribute / Method	Notes
+ getInstance() → SystemConfig [class]	Returns sole instance
+ getThreshold(tier: str) → float	Returns score boundary
+ updatePolicy(key, val) → None	Updates config; increments configVersion

Benefit: Guarantees exactly one canonical configuration object at runtime regardless of how many controllers are instantiated. Thread-safe initialisation prevents duplicate creation under concurrent Django/Celery worker load.

1.2 Structural Patterns

1.2.1 Pattern 3: Facade – Analytics Subsystem

Problem: The analytics subsystem involves multiple components: SISConnector, PatternDetector, RiskScorer, ReportGenerator, and CeleryScheduler. Controllers in the student and counsellor subsystems would need to orchestrate all of these directly, creating complex, fragile inter-subsystem coupling.

Solution: AnalyticsFacade provides a single simplified interface — runFullAnalysis(studentId), getLatestRiskProfile(studentId), generateReport(scope) — that internally coordinates all sub-components. External subsystems only depend on AnalyticsFacade.

Facade Method	Internally Coordinates
runFullAnalysis(studentId)	SISConnector → PatternDetector → RiskScorer
getLatestRiskProfile(studentId)	RiskScorer → RiskProfile repository
generateReport(scope)	RiskScorer → ReportGenerator → anonymisation pipeline

Benefit: Subsystems are decoupled from analytics internals. The ML framework can be replaced without changing any calling controller.

1.2.2 Pattern 4: Decorator – Role-Based Dashboard Panels

Problem: The base Dashboard component shows a standard at-risk student list. Different roles (Counsellor, Admin, IT Security Officer) require additional panels layered on top. Subclassing each combination would create a class explosion of 2^n subclasses.

Solution: IDashboard is the component interface. BaseDashboard is the concrete component. DashboardDecorator is an abstract wrapper. Concrete decorators (TrendAnalyticsPanel, AuditLogPanel, SessionSchedulePanel) each wrap an IDashboard and add their panel before delegating render() to the wrapped component.

Component	Type	render() behaviour
IDashboard	Interface	Abstract render() → HTML
BaseDashboard	Concrete Component	Renders at-risk student list
DashboardDecorator	Abstract Wrapper	Delegates to wrapped.render()
TrendAnalyticsPanel	Concrete Decorator	Adds KPI trend charts
AuditLogPanel	Concrete Decorator	Adds audit trail panel
SessionSchedulePanel	Concrete Decorator	Adds counsellor session schedule

Benefit: Panels can be stacked at runtime (e.g., Admin dashboard = AuditLogPanel(TrendAnalyticsPanel(BaseDashboard())) without any subclass explosion. New panels require only a new decorator class.

1.3 Behavioural Patterns

1.3.1 Pattern 5: Observer – Alert Notification

Problem: When a student's risk tier changes to High, multiple components must react independently: NotifController must dispatch alerts, AuditLogger must log the event, and DashboardRefresher must update the counsellor view. Hardcoding these reactions inside RiskProfile creates a tightly coupled dependency web that violates the Single Responsibility Principle.

Solution: RiskProfile acts as the Subject. It maintains a transient list of IRiskObserver subscribers. When the risk tier changes to High, it calls notifyObservers(), which dispatches a RiskEvent to every registered observer. NotifController, AuditLogger, and DashboardRefresher implement IRiskObserver and react independently.

Class / Interface	Role	Key Method
RiskProfile	Subject	subscribe(o), unsubscribe(o), notifyObservers()
IRiskObserver	Observer Interface	onRiskChanged(event: RiskEvent)
NotifController	Concrete Observer	onRiskChanged → dispatches alert
AuditLogger	Concrete Observer	onRiskChanged → appends AuditLog entry
DashboardRefresher	Concrete Observer	onRiskChanged → invalidates Redis cache

Benefit: New consumers (e.g., a mobile push notification service) can subscribe to RiskProfile without touching RiskProfile or existing observers.

1.3.2 Pattern 6: Strategy – Risk Scoring Algorithm

Problem: The risk scoring algorithm must be interchangeable at runtime. The admin may switch between a rule-based scorer (fast, for immediate post-submission feedback) and an ML-based scorer (accurate, for weekly batch analysis) without modifying RiskController.

Solution: Define an IRiskScoringStrategy interface with a single score(inputs: ScoringInputs): ScoringResult method. Two concrete strategies implement it: RuleBasedScorer and MLModelScorer. RiskController holds a reference to the interface and delegates scoring, enabling runtime swap via setStrategy().

Component	Type	Notes
IRiskScoringStrategy	Interface	score(inputs: ScoringInputs): ScoringResult
RuleBasedScorer	Concrete Strategy	Fast; used for immediate post-submission feedback
MLModelScorer	Concrete Strategy	– modelVersion: str; high accuracy; weekly batch
RiskController	Context	– strategy: IRiskScoringStrategy; setStrategy(s)

Benefit: New scoring algorithms (e.g., transformer-based model) can be added without modifying RiskController. Admin configures the active strategy via SystemConfig.

1.3.3 Pattern 7: State – Counselling Session Lifecycle

Problem: A CounsellingSession has five distinct lifecycle states: Requested, Confirmed, Completed, Cancelled, and NoShow. Behaviour differs significantly per state (e.g., cancel() is valid in Requested and Confirmed but not in Completed). Encoding all transitions as if/elif chains inside CounsellingSession creates an unmaintainable conditional mess.

Solution: An ISessionState interface declares all lifecycle operations (confirm(), complete(), cancel(), markNoShow()). Each concrete state class implements only the operations valid in that state; invalid operations raise InvalidTransitionException. CounsellingSession delegates all lifecycle calls to its current state object and swaps state on valid transitions.

State Class	Valid Operations	Invalid Operations
RequestedState	confirm(), cancel()	complete(), markNoShow()
ConfirmedState	complete(), cancel(), markNoShow()	confirm()
CompletedState	— (terminal)	all others → InvalidTransitionException
CancelledState	— (terminal)	all others → InvalidTransitionException
NoShowState	— (terminal)	all others → InvalidTransitionException

Benefit: Each state class is independently testable. Adding a new state (e.g., Rescheduled) requires only a new class implementing ISessionState; no changes to CounsellingSession or other states are needed.

1.3.4 Pattern 8: Command – Admin Configuration Changes

Problem: Admin operations (update risk threshold, change RBAC policy, modify retention period) must be auditable, undoable in case of misconfiguration, and executable asynchronously via Celery workers. Directly calling SystemConfig methods from a view handler prevents undo and makes async dispatch difficult.

Solution: Each admin operation is encapsulated as an ICommand object with execute() and undo() methods. UpdateRiskThresholdCommand, UpdateRBACCommand, and UpdateRetentionCommand are concrete commands that store the previous state before execution. AdminCommandInvoker maintains a command history stack enabling multi-level undo. Commands can be serialised and pushed to Celery for async execution.

Component	Type	Key Attributes / Methods
ICommand	Interface	execute() → None, undo() → None
UpdateRiskThresholdCmd	Concrete Command	– prevValue: float; execute(), undo()
UpdateRBACCmd	Concrete Command	– prevPolicy: dict; execute(), undo()
UpdateRetentionCmd	Concrete Command	– prevDays: int; execute(), undo()
AdminCommandInvoker	Invoker	– history: Stack[ICommand]; invoke(cmd), undoLast()

Benefit: Every admin configuration change is reversible, fully auditable (command object serialised to AuditLog), and asynchronously executable without coupling the admin view to SystemConfig directly.

2 MISSING ATTRIBUTES AND OPERATIONS IDENTIFIED

The following attributes and operations were absent from the Lab-07 analysis model. They have been identified via design-pattern integration, contract specification, and cross-checking with the SDD (Part A) entity and subsystem tables.

Missing Attributes

Class	Attribute	Type	Reason Added
User	failedAttempts	int	5-attempt lockout logic in AuthController
User	twoFAEnabled	bool	Mandatory True for Admin/ITOfficer; optional for Student
User	lastLoginAt	Optional[datetime]	Session audit and inactivity detection
User	createdAt	datetime	Immutable creation timestamp; required by audit trail
ScreeningRecord	isComplete	bool	Distinguishes partial saves from full submissions
ScreeningRecord	version	int	Auto-incremented per (studentId, formType) for versioning
ScreeningRecord	formType	FormTypeEnum	Required to apply correct scoring rules per form
MoodLog	sleepQuality	int	1–5 Likert scale; separate from sleepHrs for richer signal
MoodLog	dataFedAt	Optional[datetime]	Timestamp when log was consumed by risk algorithm
RiskProfile	observers	List[IRiskObserver]	Required by Observer pattern; transient, not persisted
RiskProfile	alertSent	bool	Prevents duplicate High-risk alerts for same tier event
RiskProfile	alertSentAt	Optional[datetime]	Timestamp of last dispatched alert; suppression logic
RiskController	strategy	IRiskScoringStrategy	Required by Strategy pattern; injected at runtime
PatternReport	modelVersion	str	Traceability: ML model version that produced report
PatternReport	confidence	float	Model confidence score (0.0–1.0); fed into composite scorer
StudentAcadData	syncVersion	int	Detects stale SIS data; incremented on each successful sync
StudentAcadData	sisTimestamp	datetime	Exact SIS export timestamp for incremental sync logic
NotifController	factory	NotificationFactory	Required by Factory Method pattern

Missing Operations

Class	Missing Operation	Reason Added
AuthController	lockAccount(userId)	Required after 5 failed logins; missing in Lab-07
AuthController	unlockAccount(userId)	Admin-triggered unlock; resets locked + failedAttempts
AuthController	incrementFailedAttempts(userId)	Private; increments counter and triggers lock at 5
User	logout()	Invalidates session token in Redis session store
User	resetPassword(newPwd)	Hashes and stores new password; resets failedAttempts = 0
ScreeningRecord	computeScore()	Derives totalScore from responses using clinical rules
ScreeningRecord	validate()	Returns bool; checks all required questions answered
ScreeningRecord	partialSave()	Persists incomplete state; sets isComplete = False
RiskProfile	subscribe(o: IRiskObserver)	Observer pattern: register new observer
RiskProfile	unsubscribe(o: IRiskObserver)	Observer pattern: deregister observer
RiskProfile	notifyObservers()	Observer pattern: broadcast tier change event to all observers
RiskController	setStrategy(s)	Strategy pattern: swap scoring algorithm at runtime
RiskController	loadInputs(studentId)	Private; aggregates all data sources into ScoringInputs
AnalyticsFacade	runFullAnalysis(studentId)	Facade: orchestrates all analytics pipeline steps
AnalyticsFacade	getLatestRiskProfile(id)	Facade: single-call risk retrieval for external controllers
AnalyticsFacade	generateReport(scope)	Facade: produces anonymised institutional reports
NotifController	createNotification(type, ctx)	Factory Method: delegates creation to NotificationFactory
NotifRecord	retryDelivery()	Requeues failed notification; increments retryCount
CounsellingSession	setState(s: ISessionState)	State pattern: transitions session to new state
CounsellingSession	classify()	Delegates classify call to current state

3 TYPE, SIGNATURES, AND VISIBILITY SPECIFICATION

Full specifications for all analysis classes across every subsystem. Visibility: + public, – private, # protected. Aligned with SDD (Part A) entity definitions throughout.

Class: User (Abstract Entity)

Vis	Attribute / Method	Type / Signature	Notes
–	userId	str	UUID; PK; immutable after creation
–	email	str	Institutional email; unique index
–	passwordHash	str	bcrypt(12) hash
–	role	RoleEnum	{Student, Counsellor, Admin, Guardian, ITOfficer}
–	locked	bool	Default False; set True after 5 failed logins
–	failedAttempts	int	Default 0; reset on successful login
–	twoFAEnabled	bool	Mandatory True for Admin/ITOfficer
–	createdAt	datetime	UTC; immutable
–	lastLoginAt	Optional[datetime]	Updated each successful authentication
+	authenticate(pwd: str) → bool	method	Abstract; subclass verifies credentials
+	logout() → None	method	Invalidates session token in Redis
+	resetPassword(newPwd: str) → None	method	Hashes and stores; resets failedAttempts = 0

Class: AuthController (Control)

Vis	Attribute / Method	Type / Signature	Notes
–	userRepo	IUserRepository	Injected dependency
–	sessionStore	ISessionStore	Redis-backed session manager
+	login(email: str, pwd: str) → Token	method	Returns JWT; raises AuthError on failure
+	verifyTwoFA(userId: str, code: str) → bool	method	Validates TOTP; returns bool
+	lockAccount(userId: str) → None	method	Sets User.locked = True; logs to AuditLog
+	unlockAccount(userId: str) → None	method	Admin-only; resets locked and failedAttempts
+	logout(token: str) → None	method	Invalidates session token in Redis
–	incrementFailedAttempts(userId: str) → None	method	Private; locks if failedAttempts >= 5

Class: ScreeningRecord (Entity)

Vis	Attribute / Method	Type / Signature	Notes
–	recordId	str	UUID; PK
–	studentId	str	FK → User.userId
–	formType	FormTypeEnum	{PHQ9, GAD7, PSS}
–	responses	dict	JSON: {questionId: answerInt}
–	totalScore	int	Derived by computeScore()
–	riskTier	RiskEnum	Derived from totalScore per clinical thresholds
–	version	int	Strictly increasing per (studentId, formType)
–	submittedAt	datetime	UTC; immutable after isComplete = True
–	isComplete	bool	False during partial-save; True after full submission
+	computeScore() → int	method	Sums weighted responses per clinical scoring rules
+	validate() → bool	method	Checks all required questions answered
+	partialSave() → None	method	Persists incomplete state; sets isComplete = False

Class: MoodLog (Entity)

Vis	Attribute / Method	Type / Signature	Notes
–	logId	str	UUID; PK
–	studentId	str	FK → User.userId
–	moodScore	int	1–10 Likert scale; required
–	sleepHrs	float	Hours of sleep; range 0.0–24.0
–	sleepQuality	int	1–5 Likert scale; separate from duration
–	stressors	str	Free text; optional; max 500 chars
–	logDate	date	UTC date; one log per student per day enforced
–	dataFedAt	Optional[datetime]	Timestamp when consumed by risk algorithm
+	save() → None	method	Persists log; triggers risk algorithm feed
+	validate() → bool	method	Checks moodScore in [1,10] and sleepHrs in [0,24]

Class: RiskProfile (Entity)

Vis	Attribute / Method	Type / Signature	Notes
–	profileId	str	UUID; PK
–	studentId	str	FK → User.userId; unique
–	compositeScore	float	Range 0.0–100.0; derived
–	riskTier	RiskEnum	{Low, Medium, High}; derived from compositeScore
–	trendData	JSON	List of {date, score} pairs; max 52 entries
–	classifiedAt	datetime	UTC timestamp of last scoring run
–	alertSent	bool	Default False; reset when tier drops below High
–	alertSentAt	Optional[datetime]	Timestamp of last dispatched High-risk alert
–	observers	List[IRiskObserver]	Transient; not persisted to DB
+	subscribe(o: IRiskObserver) → None	method	Adds observer to transient list
+	unsubscribe(o: IRiskObserver) → None	method	Removes observer from list
–	notifyObservers() → None	method	Private; dispatches RiskEvent to all observers
+	updateTier(score: float) → None	method	Sets riskTier; calls notifyObservers if tier changed
+	getLatest() → RiskProfile	method	Returns most recent profile for studentId

Class: RiskController (Control)

Vis	Attribute / Method	Type / Signature	Notes
–	strategy	IRiskScoringStrategy	Injected; default: RuleBasedScorer
–	profileRepo	IRiskProfileRepository	Injected repository
+	setStrategy(s: IRiskScoringStrategy) → None	method	Swaps scoring algorithm at runtime
+	classifyRisk(studentId: str) → RiskTier	method	Loads inputs, delegates to strategy, updates profile
+	getProfile(studentId: str) → RiskProfile	method	Retrieves latest profile via repository
–	loadInputs(studentId: str) → ScoringInputs	method	Private; aggregates all data sources

Class: NotifController (Control)

Vis	Attribute / Method	Type / Signature	Notes
–	factory	NotificationFactory	Required by Factory Method pattern; injected
–	notifRepo	INotifLogRepository	Injected repository for delivery logging
+	createNotification(type, ctx) → Notification	method	Delegates to factory; returns correct subtype
+	sendInApp(userId, msg) → None	method	Pushes to in-app notification queue
+	sendEmail(addr, msg) → None	method	Dispatches via email service; logs delivery
+	logDelivery(notifId, status) → None	method	Persists delivery outcome to NotifRecord
+	retryQueue(notifId) → None	method	Requeues failed delivery; increments retryCount
+	onRiskChanged(event: RiskEvent) → None	method	Observer: triggers alert notification pipeline

Class: CounsellingSession (Entity)

Vis	Attribute / Method	Type / Signature	Notes
–	sessionId	str	UUID; PK
–	studentId	str	FK → User.userId
–	counsellorId	str	FK → User.userId
–	slotDateTime	datetime	UTC; must be in the future at creation
–	currentState	ISessionState	State pattern: current lifecycle state object
–	reminderSent	bool	Idempotency flag; prevents duplicate reminders
–	sessionNotes	Optional[str]	Encrypted; non-null iff status = Completed
–	cancellationNote	Optional[str]	Non-null iff status = Cancelled
–	createdAt	datetime	UTC; immutable
+	setState(s: ISessionState) → None	method	State pattern: transitions to new state
+	confirm() → None	method	Delegates to currentState.confirm()
+	complete(notes: str) → None	method	Delegates to currentState.complete()
+	cancel(note: str) → None	method	Delegates to currentState.cancel()
+	save() → None	method	Persists session via repository

Class: ConsentRecord (Entity)

Vis	Attribute / Method	Type / Signature	Notes
–	consentId	str	UUID; PK
–	studentId	str	FK → User.userId
–	guardianId	str	FK → User.userId
–	granted	bool	True = active consent
–	grantedAt	datetime	UTC; immutable after creation
–	revokedAt	Optional[datetime]	Null if active; set on revocation
–	anonymisationQueued	bool	True after revocation queues anonymisation
+	revoke(revokedAt: datetime) → None	method	Sets revocation timestamp; queues anonymisation
+	isActive() → bool	method	Returns True iff granted = True and revokedAt is null
+	query() → ConsentRecord	method	Retrieves record for (studentId, guardianId) pair

Class: SystemConfig (Singleton)

Vis	Attribute / Method	Type / Signature	Notes
–	_instance	SystemConfig	Class-level Singleton holder; default None
–	riskThresholds	dict	{Low:40, Medium:70, High:100}; configurable
–	retentionDays	int	Data retention period; default 730 (2 years)
–	rbacPolicies	dict	Role → permission mapping
–	configVersion	int	Incremented on every policy change
–	_lock	RLock	Reentrant lock for thread-safe access
+	getInstance() → SystemConfig	class method	Returns sole instance; thread-safe
+	getThreshold(tier: str) → float	method	Returns score boundary for given risk tier
+	updatePolicy(key: str, val) → None	method	Updates config; increments configVersion

Class: AuditLog (Entity)

Vis	Attribute / Method	Type / Signature	Notes
–	logId	str	UUID; PK; append-only
–	actorId	str	FK → User.userId; 'SYSTEM' for automated events
–	actorRole	RoleEnum	Role at time of action; stored for RBAC audit
–	actionType	str	E.g. Login, RiskClassified, AlertSent, ConsentRevoked
–	targetId	Optional[str]	ID of affected entity
–	timestamp	datetime	UTC; strictly monotonically increasing per session
–	hashChain	str	HMAC-SHA256 of (prev_hash entry data); 64-char hex
+	computeHashChain(prevHash: str) → str	method	Generates HMAC; stored in hashChain
+	verifyChain(fromEntry: int) → bool	method	Validates integrity of all entries from index forward

4 PRECONDITIONS AND POSTCONDITIONS

All contracts use Design by Contract (DbC) notation. Pre: conditions that must hold before invocation. Post: guaranteed state after successful return. Exc: guaranteed behaviour on failure. All time references are UTC.

[Auth] AuthController.login(email: str, pwd: str) → Token

Contract	
Pre:	email is non-null and matches institutional email regex. pwd is non-null and >= 8 characters. User with email exists. User.locked = False.
Post:	Valid JWT returned. User.lastLoginAt = current UTC. User.failedAttempts = 0. AuditLog entry (Login) appended.
Exc:	Wrong password: failedAttempts++; AuthError raised. failedAttempts = 5: lockAccount() called; AccountLockedError raised.

[Auth] AuthController.verifyTwoFA(userId: str, code: str) → bool

Contract	
Pre:	userId is valid and corresponds to an active User. code is a 6-digit TOTP string. Primary password authentication has already passed in the same request.
Post:	Returns True iff code is valid within the 30-second TOTP window. AuditLog entry (2FASuccess or 2FAFailed) appended.
Exc:	Invalid code: returns False; failedAttempts incremented. Expired code: returns False; user prompted to retry.

[Screening] ScreeningRecord.computeScore() → int

Contract	
Pre:	responses is non-null and non-empty. formType is a valid FormTypeEnum value. All response values are integers within the valid range for the formType (PHQ-9: 0–3 per question; GAD-7: 0–3; PSS: 0–4).
Post:	Returns integer score in valid range (PHQ-9: 0–27; GAD-7: 0–21; PSS: 0–40). totalScore attribute set. riskTier derived and set per clinical thresholds.
Exc:	Any response out of range: ValidationError raised; no score computed; record stays in partial-save state.

[Screening] ScreeningRecord.partialSave() → None

Contract	
Pre:	Student is authenticated. At least one response has been entered. isComplete = False.
Post:	Current response state persisted to DB. isComplete remains False. No score computed. submittedAt remains null.

[Risk] RiskProfile.updateTier(score: float) → None

Contract	
----------	--

Pre:	score is in [0.0, 100.0]. studentId references an active Student.
Post:	compositeScore updated to score. riskTier set per SystemConfig thresholds. If tier changed, notifyObservers() called exactly once. If new tier = High and alertSent = False: alertSent = True, alertSentAt = current UTC.
Exc:	score out of range: ValueError raised; no state change.

[Counselling] CounsellingSession.setState(s: ISessionState) → None

Contract	
Pre:	s is a valid concrete ISessionState. The transition from currentState to s is permitted by the state machine (see invariant §5.1).
Post:	currentState = s. Session persisted via save(). AuditLog entry appended.
Exc:	Invalid transition: InvalidTransitionException raised; currentState unchanged.

5 INVARIANTS

5.1 Class Invariants

#	Invariant — User
1	userId is immutable: no operation modifies userId after createdAt is set
2	failedAttempts ≥ 0 at all times
3	locked = True $\Rightarrow$ failedAttempts ≥ 5
4	role in {Admin, ITOfficer} $\Rightarrow$ twoFAEnabled = True
5	email matches institutional regex: <code>/[a-z0-9.]+@[institution]\$/</code>
6	lastLoginAt != null $\Rightarrow$ lastLoginAt $\geq$ createdAt

#	Invariant — RiskProfile
1	Exactly one RiskProfile per active Student: for all s in Students, $ \{p \mid p.\text{studentId} = s.\text{userId}\} = 1$
2	compositeScore in [0.0, 100.0] at all times
3	riskTier = Low $\Leftrightarrow$ compositeScore < 40
4	riskTier = Medium $\Leftrightarrow$ compositeScore in [40, 70)
5	riskTier = High $\Leftrightarrow$ compositeScore ≥ 70
6	alertSent = True $\Rightarrow$ riskTier = High
7	alertSentAt != null $\Leftrightarrow$ alertSent = True
8	classifiedAt $\leq$ currentUTC always

#	Invariant — ScreeningRecord
1	isComplete = True $\Rightarrow$ totalScore is set AND riskTier is derived
2	submittedAt is immutable once isComplete = True
3	formType = PHQ9 $\Rightarrow$ totalScore in [0, 27]
4	formType = GAD7 $\Rightarrow$ totalScore in [0, 21]
5	formType = PSS $\Rightarrow$ totalScore in [0, 40]
6	version ≥ 1 and strictly increasing per (studentId, formType) pair

#	Invariant — MoodLog
1	moodScore in [1, 10] at all times
2	sleepHrs in [0.0, 24.0] at all times
3	sleepQuality in [1, 5] at all times
4	At most one MoodLog per (studentId, logDate) pair

#	Invariant — <i>CounselingSession</i>
1	Valid state transitions only: Requested -> Confirmed -> {Completed, NoShow, Cancelled}; Requested -> Cancelled
2	cancellationNote != null <=> currentState = CancelledState
3	sessionNotes != null <=> currentState = CompletedState
4	slotDateTime > createdAt always
5	No two Confirmed sessions share the same (counselorId, slotDateTime) pair

#	Invariant — <i>ConsentRecord</i>
1	granted = False => all guardian-facing endpoints return HTTP 403 for this student
2	revokedAt = null <=> granted = True
3	revokedAt != null => revokedAt >= grantedAt
4	At most one active (granted = True) ConsentRecord per (studentId, guardianId) pair

5.2 Method-Level Invariants

RiskProfile.updateTier(score: float)

In va ri ant	Detail — <i>RiskProfile.updateTier(score: float)</i>
•	Before and after: compositeScore in [0.0, 100.0]
•	Before and after: riskTier is consistent with compositeScore per tier thresholds
•	After: If tier changed, notifyObservers() has been called exactly once
•	After: If tier changed to High and alertSent = False: alertSent = True and alertSentAt is set

AuditLog.computeHashChain(prevHash: str) → str

In va ri ant	Detail — <i>AuditLog.computeHashChain(prevHash: str) → str</i>
•	Before: prevHash is a valid 64-character lowercase hex string
•	After: Returns a valid 64-character hex string
•	Deterministic: Same (prevHash, entry data) always produces the same output
•	Cascade invariant: Modifying any past entry invalidates all subsequent hashChain values, making tampering detectable on verifyChain() call

AuthController.incrementFailedAttempts(userId: str) → None

invariant	Detail — AuthController.incrementFailedAttempts(userId: str) → None
•	Before: User.failedAttempts >= 0
•	After: User.failedAttempts_after = User.failedAttempts_before + 1
•	After: If failedAttempts_after >= 5: User.locked = True and lockAccount() was called exactly once
•	Monotonicity: failedAttempts never decreases during a failed authentication sequence; only reset on successful login

AdminCommandInvoker.undoLast() → None

invariant	Detail — AdminCommandInvoker.undoLast() → None
•	Before: history stack is non-empty
•	After: The most recently invoked command's undo() has been called exactly once
•	After: That command has been popped from history; stack size decremented by 1
•	After: SystemConfig.configVersion incremented (undo is itself a versioned change)
•	Idempotency excluded: Calling undoLast() twice undoes two commands, not the same one

CounsellingSession.setState(s: ISessionState) → None

invariant	Detail — CounsellingSession.setState(s: ISessionState) → None
•	Before: s is a valid concrete ISessionState and the transition from currentState to s is permitted by the state machine
•	After: currentState = s
•	Invariant preserved: currentState is always a non-null ISessionState instance; never null mid-transition

5.3 System-Wide Invariants

#	Name	Invariant Statement
1	RBAC Completeness	Every HTTP request that reaches a protected endpoint is preceded by a successful JWT validation. No endpoint is reachable without a valid session token in any operational state.
2	Risk-Alert Consistency	At no point does a student have RiskProfile.riskTier = High and alertSent = False for longer than 2 minutes under normal system operation (alert latency SLA).
3	Consent Gate	For all API calls c with actorRole = Guardian: if ConsentRecord.isActive() returns False for the targeted student, c returns HTTP 403 and no student data is exposed in the response body.

#	Name	Invariant Statement
4	Audit Completeness	For all write operations w on any entity in {User, RiskProfile, CounsellingSession, ConsentRecord, SystemConfig}: an AuditLog entry with a valid hashChain exists for w before the HTTP response is returned to the caller.
5	Singleton Uniqueness	$ \{\text{SystemConfig}\}_{\text{instance}} = 1$ for the entire lifetime of the application process, regardless of the number of concurrent Django workers or Celery threads.
6	Data Minimisation (GDPR)	No admin-level report export contains a field that uniquely identifies an individual student. All exports pass through the anonymisation pipeline before being made available for download.
7	Score-Tier Coherence	For all RiskProfile p in the database: the stored riskTier is the correct tier for the stored compositeScore under the current SystemConfig thresholds (enforced by reclassification on every threshold change).
8	Hash Chain Integrity	For all $n > 1$: $\text{AuditLog}[n].\text{hashChain} = \text{HMAC-SHA256}(\text{AuditLog}[n-1].\text{hashChain} \text{AuditLog}[n].\text{data})$. Any violation detected by <code>verifyChain()</code> triggers an immediate IT Security Officer alert.
9	Session Non-Overlap	No counsellor has two sessions with <code>currentState = ConfirmedState</code> that share the same <code>slotDateTime</code> . Enforced by a unique database constraint on (<code>counsellorId</code> , <code>slotDateTime</code> , <code>status = Confirmed</code>).
10	Notification Frequency Cap	No user receives more than 3 automated notifications of the same type within any 24-hour rolling window. Enforced by <code>NotifController</code> before each dispatch.

6 CONCLUSION

This Object Design Document completes the transition from analysis-level models to a fully specified, implementation-ready design for the Student Mental Health Monitoring System.

This document applied eight GoF and architectural design patterns across three categories to resolve concrete engineering challenges in SMHMS. The Observer pattern has been correctly classified as Behavioural (consistent with the SDD control flow table, Section 6.1), and all class and attribute names are aligned with the SDD entity and subsystem definitions.

Pattern Category Summary

Category	Patterns Applied	Primary Concern Addressed
Creational	Factory Method (1), Singleton (2)	Controlled object creation; configuration consistency
Structural	Facade (3), Decorator (4)	Subsystem decoupling; runtime panel composition
Behavioural	Observer (5), Strategy (6), State (7), Command (8)	Event propagation; algorithm selection; lifecycle; auditability

SOLID Principle Alignment

Principle	Application in SMHMS
Single Responsibility	RiskProfile models risk state only; alerting is in NotifController; scoring is in IRiskScoringStrategy
Open / Closed	Abstract hierarchies (User, Notification, IDashboard, ICommand) allow new types by subclassing without modifying existing classes
Liskov Substitution	All concrete Users replace User in any authentication context. All ISessionState implementations are interchangeable in CounsellingSession
Interface Segregation	Repository interfaces are narrow and role-specific; no client is forced to depend on methods it does not use
Dependency Inversion	All controllers depend on repository interfaces, strategy interfaces, and factory interfaces — never on concrete implementations
Design by Contract	All operations in Section 4 carry explicit preconditions, postconditions, and exception guarantees sufficient for TDD

By the end of this ODD, every class in SMHMS has complete attribute specifications (Section 3), complete method contracts (Section 4), and invariant coverage (Section 5) specifying 35+ invariants at class, method, and system level. Every design decision references a named GoF pattern with problem statement and benefit, enabling future maintainers to extend the design without reverse-engineering.

The system is ready to proceed to the implementation phase with Django/PostgreSQL/Celery, with all interface contracts, repository boundaries, and pattern scaffolding fully defined and consistent with the System Design Document (Part A).